

Comparação Entre Algoritmos de Escalonamento

Breno C. R. Nakamura¹, Estevan T. Santos¹, Gabriel L. Bonavina¹,
Jonas L. Durão¹, Natália V. A. do Val¹, Vinicius A. L. Andrade¹

¹Instituto de Ciência e Tecnologia – Universidade Federal de São Paulo (UNIFESP)
Av. Cesare Monsueto Giulio Lattes, 1201 – 12247-014 – São José dos Campos, SP

Abstract. *This meta-paper describes the style to be used in articles and short papers for SBC conferences. For papers in English, you should add just an abstract while for the papers in Portuguese, we also ask for an abstract in Portuguese (“resumo”). In both cases, abstracts should not have more than 10 lines and must be in the first page of the paper.*

Resumo. *Este meta-artigo descreve o estilo a ser usado na confecção de artigos e resumos de artigos para publicação nos anais das conferências organizadas pela SBC. É solicitada a escrita de resumo e abstract apenas para os artigos escritos em português. Artigos em inglês deverão apresentar apenas abstract. Nos dois casos, o autor deve tomar cuidado para que o resumo (e o abstract) não ultrapassem 10 linhas cada, sendo que ambos devem estar na primeira página do artigo.*

1. Introdução

2. Instalação

3. Mudança nos Banners

4. Mudança `exec.c`

5. Descrição do Exercício Proposto

6. Descrição dos Algoritmos de Escalonamento

6.1. Algoritmo Padrão do MINIX

6.2. Round-Robin

O algoritmo de escalonamento por chaveamento circular (ou *Round-Robin*), é um dos algoritmos mais simples e antigos devido à sua fácil implementação. Seu funcionamento gira em torno da definição de uma fatia de tempo de CPU (*quantum*), em que o processo que está na CPU terá um tempo limitado de uso e, ao final deste tempo, haverá preempção e um novo processo será escolhido para ser executado [Tanenbaum 2016].

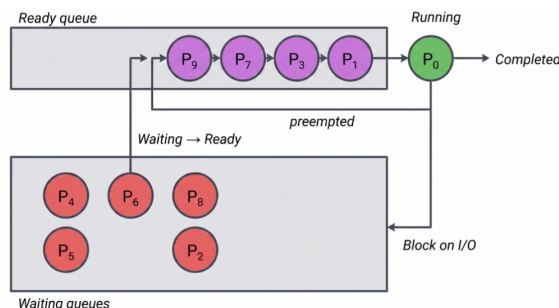


Figura 1. Diagrama de funcionamento do *Round-Robin* [da Costa 2026]

Para implementá-lo, é necessária a utilização de uma fila circular de processos prontos. Para nossa implementação, substituímos as múltiplas filas do algoritmo padrão do MINIX por uma fila única de processos de usuário (USER_Q 7), além disso, desabilitamos os algoritmos de demora, promoções periódicas de prioridade e rebalanceamento de filas. Por fim, modificamos o *quantum* para que ele seja variável, com a seguinte lógica: cada processo terá uma fatia de CPU inversamente proporcional ao número de processos de usuário prontos. Em sua fila circular, quando o *quantum* do processo em execução acabar, ele deverá ser encaminhado para o final da fila. Do mesmo modo, quando ele for bloqueado por entrada e saída, quando seu estado for atualizado para pronto, ele deverá retornar ao final da fila, como pode ser visto na Figura 1.

6.3. Múltiplas Filas

6.4. Algoritmo a ser escolhido

7. Resultados Obtidos e Discussão

Referências

da Costa, J. B. D. (2026). Capítulo 2 - processos e threads. Slides apresentados na disciplina de Sistemas Operacionais.

Tanembaum, A. S. (2016). *Sistemas Operacionais Modernos*. São Paulo: Pearson Education Brasil, 4th edition.